EXPERIMENT 14

Exploring Pro-Code Extension Using Zoho Catalyst and Custom Functions

Recruitment & Job Application Management — Candidate Review Scoring Extension

Student Name	Lalitha B
Roll Number	241401045
Department	Computer Science and Business Systems (CSBS)
Platform	Zoho Creator
Application	Recruitment & Job Application Management
Experiment Number	14

1. Aim

To explore how the existing Recruitment & Job Application Management application can be extended beyond standard low-code workflow actions by combining a reusable Deluge custom function with a proposed Zoho Catalyst Advanced I/O serverless function. The case study examines a demonstration candidate review-priority calculation, integration design, and associated security and reliability precautions.

2. Existing Application and Extension Need

The application developed in earlier experiments includes the Job Application Form, All Job Applications report, automated acknowledgement workflow, Zoho Calendar integration design, recruitment analytics dashboard and HR-role permissions. Basic form submission and notifications fit within Zoho Creator. A reusable extension becomes relevant when rules must be centralized or specialized external processing is required.

Chosen conceptual use case: Assign an illustrative review category (High, Medium or Normal) using supplied experience, skills and department. This is a technical demonstration only, not a hiring recommendation; real decisions need role-specific, validated criteria and human review.

3. Procedure

Step 1. Review existing Job Application Form and earlier workflows; identify experience, skills and department as potential inputs.

Step 2. Compare built-in workflow actions, Deluge functions, Creator Custom API and Catalyst functions.

Step 3. Design the Creator-to-Catalyst architecture with an authenticated HTTPS connection.

Step 4. Draft a reusable Deluge function that prepares JSON and calls an external endpoint.

Step 5. Draft a Node.js Advanced I/O handler that validates inputs and returns a demonstration category.

Step 6. Describe normal and error paths from form submission to optional record update.

Step 7. Document security, failure handling, deployment considerations and maintainability.

4. Evidence from the Existing Zoho Creator Application

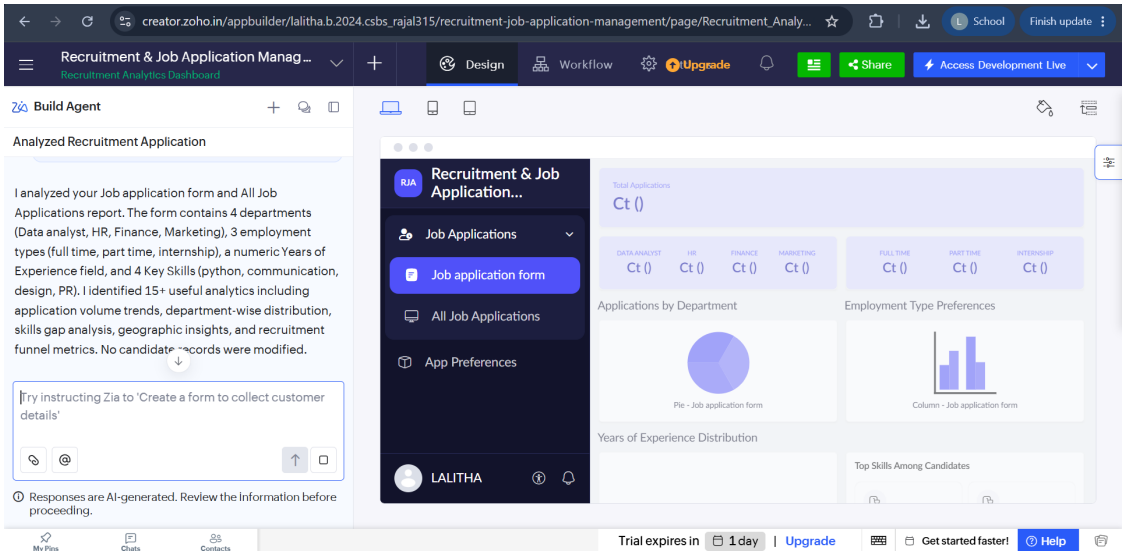


Figure 1. Earlier screenshot showing the Recruitment Analytics Dashboard and the existing Job Application Form menu.

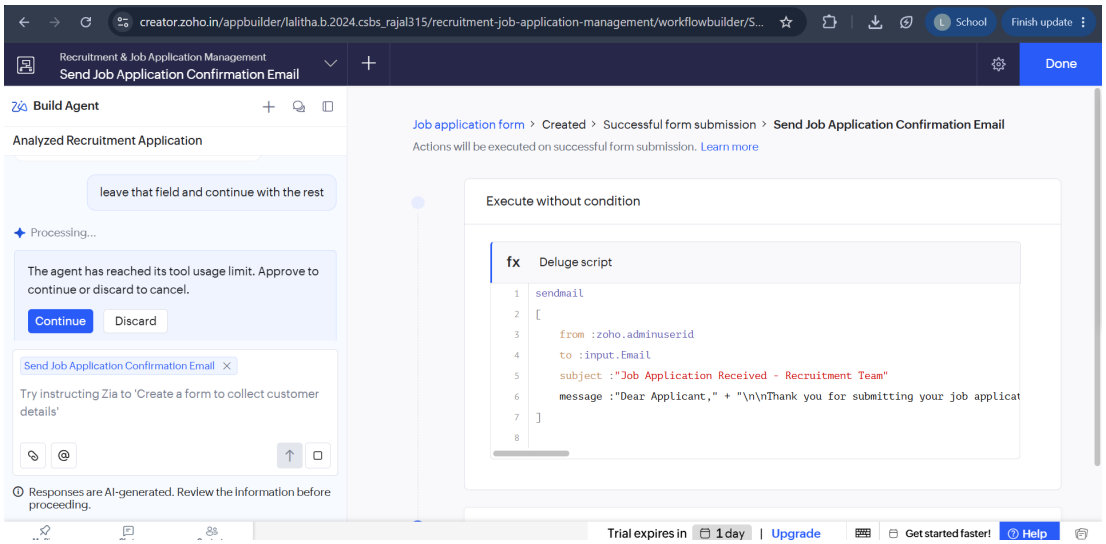


Figure 2. Existing “Send Job Application Confirmation Email” workflow and Deluge action, created in an earlier experiment.

5. Proposed Pro-Code Extension Architecture

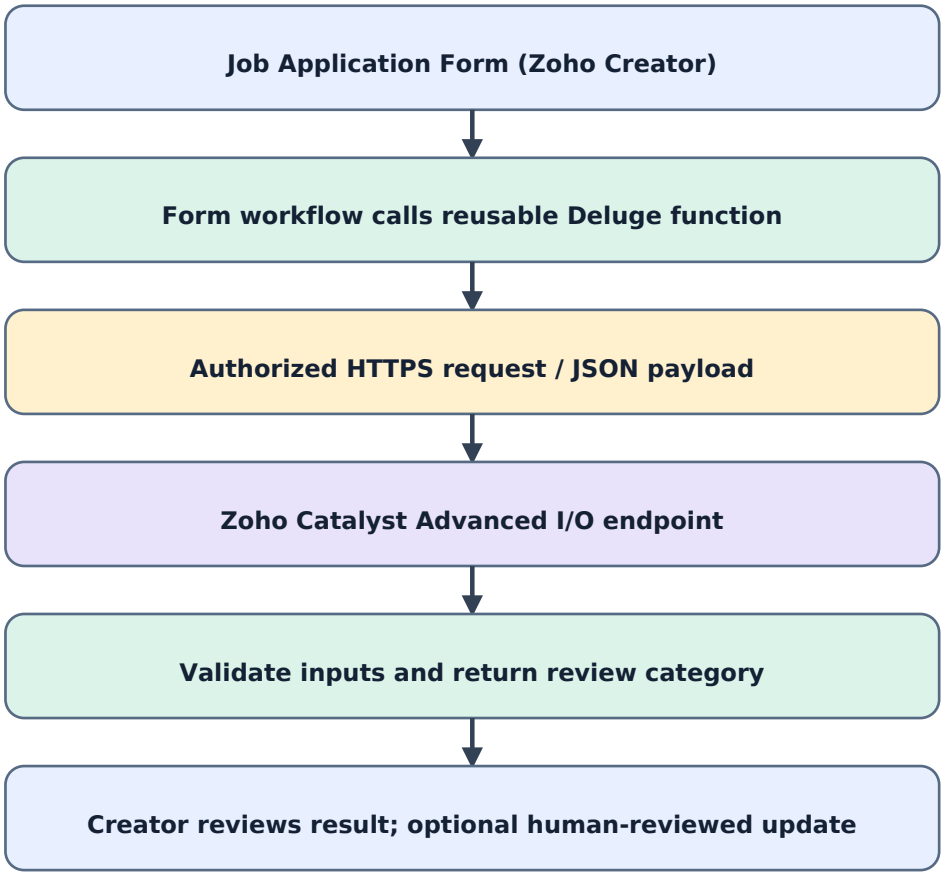


Figure 3. Conceptual architecture only; the Catalyst endpoint and Creator connection have not been deployed or tested.

6. Comparison of Extension Methods

Option	What it does	Suitable recruitment task
Workflow actions	Built-in event-triggered actions	Submission acknowledgements and basic updates
Deluge custom function	Reusable function within Creator	Shared candidate-field validation and rules
Creator Custom API	Creator-exposed service endpoint	External HR system invokes authorized Creator logic
Catalyst Advanced I/O	Pro-code function invoked over HTTPS	Specialized algorithms or external libraries

7. Illustrative Deluge Custom Function

The following pseudocode-style Deluge function body demonstrates how Creator could prepare the request. The endpoint and connection identifiers are placeholders; it is not executable as-is and should not be treated as an installed integration.

```
// Conceptual function body: getCandidateReviewCategory
payload = Map();
payload.put("experience", experience);
payload.put("skills", skills);
payload.put("department", department);
response = invokeurl
[
    url : "https://YOUR-CATALYST-ENDPOINT/score"
    type : POST
    parameters : payload.toString()
    headers : {"Content-Type":"application/json"}
    connection : "catalyst_connection"
];
return response.get("priority");
```

8. Illustrative Catalyst Function (Node.js)

A conceptual Advanced I/O function could parse a validated JSON request and respond with a category. The abbreviated handler below illustrates the logic; deployment requires the correct Catalyst runtime, endpoint configuration and authorization.

```
app.post("/score", (req, res) => {
  const { experience, skills, department } = req.body;
  if (!Number.isFinite(experience) || experience < 0 ||
      typeof skills !== "string" ||
      typeof department !== "string") {
    return res.status(400).json({error:"Invalid input"});
  }
  let score = experience >= 5 ? 2 : experience >= 2 ? 1 : 0;
  if (skills.toLowerCase().includes("python")) score += 1;
  const priority = score >= 3 ? "High" :
                   score >= 1 ? "Medium" : "Normal";
  return res.json({priority});
});
```

Illustrative cases: 6 years with Python = High; 3 years with communication skills = Medium; 0 years with design skills = Normal. Scores are arbitrary and are not appropriate for real candidate ranking.

9. Proposed Execution Flow

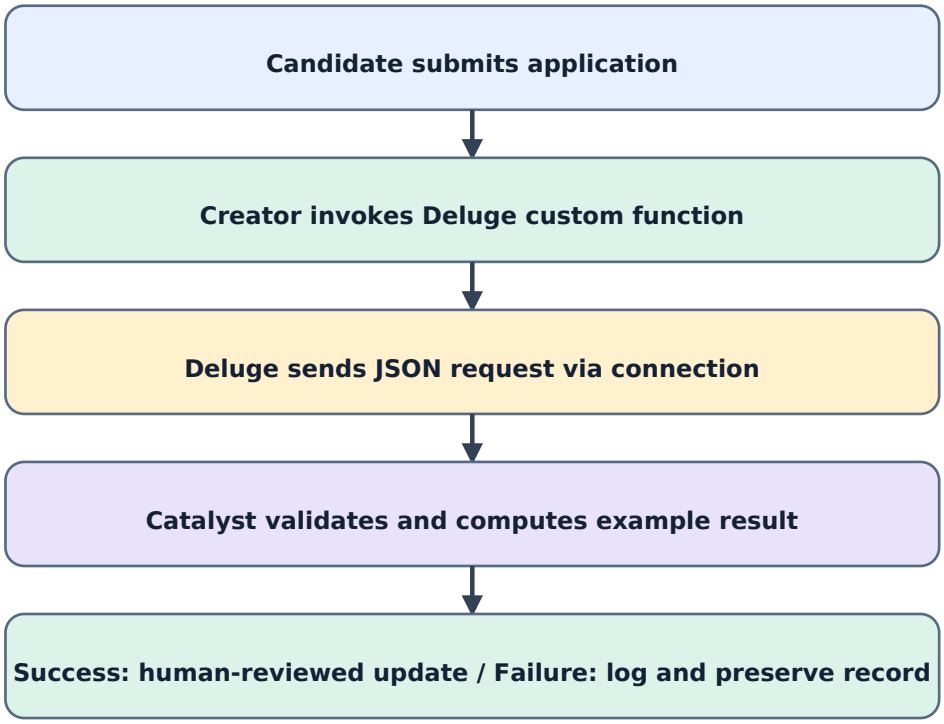


Figure 4. Proposed execution flow; failure must not prevent the original application from being retained.

10. Decision Guide

Requirement	Recommended method
Standard form, notification, or simple update	Built-in Creator workflow
Reusable logic across multiple forms	Deluge custom function
External program calling Creator operations	Creator Custom API
Specialized libraries or computationally intensive rules	Catalyst serverless function
No justified extension requirement	Keep the simpler low-code implementation

11. Benefits, Precautions and Governance

Area	Expected benefit	Precaution
Reusability	Consistent logic across workflows	Version and document function contracts
Flexibility	Use server-side libraries	Testing and code ownership required
Security	Credentials handled through connections	Least-privilege access; no secrets in source
Reliability	Separate optional processing	Timeout handling, retries and meaningful failure logs
Cost	Use on-demand processing	Check platform limits, quotas and billing

Area	Expected benefit	Precaution
Responsible use	Human review of proposed categories	Avoid unvalidated or discriminatory hiring scores

12. Result and Conclusion

This experiment explored a pro-code extension for the existing Recruitment & Job Application Management application through a reusable Deluge function and a proposed Catalyst Advanced I/O function. The architecture, sample scripts, execution flow, extension-method comparison and precautions were documented using existing application screenshots as context.

Implementation status: The existing form and email workflow are evidenced by earlier screenshots. The proposed scoring function, Catalyst endpoint and associated connection were not deployed, invoked or benchmarked. The study therefore demonstrates the design approach rather than a successfully executed external integration.